ASSIGNMENT 6

Implementation of Data Integrity Constraints

PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK and DEFAULT

1. Objective

Data integrity means that the data in the database is accurate, consistent and follows the business rules of the application. Assignment 5 created the tables with primary keys, foreign keys and NOT NULL rules. This assignment adds the remaining UNIQUE, CHECK and DEFAULT constraints (plus the composite foreign key proposed in Assignment 3), explains the purpose of every constraint used, and proves with executable SQL that invalid data is rejected and relationships are preserved.

2. Purpose of the Constraints Used

Constraint	Purpose	Where used in this database
PRIMARY KEY	Uniquely identifies each row; cannot be NULL or duplicated (entity integrity)	Every table: customer_id, pincode, address_id, category_id, product_id, cart_id, cart_item_id, order_id, order_item_id, payment_id, shipment_id, review_id
FOREIGN KEY	Ensures a child row always refers to an existing parent row (referential integrity); defines ON DELETE behaviour	14 relationships (Assignment 5) + composite FK Orders(address_id, customer_id) → Addresses
Composite FOREIGN KEY	Ensures an order's delivery address belongs to the customer who placed the order	fk_orders_address_customer
UNIQUE	Prevents duplicate values in a candidate key; also implements 1:1 relationships	email; category_name; (product_name, brand); one cart per customer; (cart_id, product_id); (order_id, product_id); one payment and one shipment per order; tracking_number; one review per customer per product
CHECK	Restricts values to a valid domain (business rule)	price > 0, stock >= 0, quantity > 0, rating 1–5, status / method lists, e-mail / phone / PIN format, expected delivery >= shipped date, totals >= 0
DEFAULT	Supplies a sensible value when none is given	registration_date, created_date, review_date = today; order_date = now; order_status = 'Placed'; payment_status = 'Pending'; shipment_status = 'In Transit'; address_type = 'Home'; stock_quantity = 0; cart quantity = 1
NOT NULL	Makes a column mandatory	All required columns (Assignment 5)

3. Constraint Script (ALTER TABLE)

Header

```
-- ASSIGNMENT 6: DATA INTEGRITY CONSTRAINTS (UNIQUE, CHECK, DEFAULT, composite FK)
USE online_shopping;
```

Customers

```
-- Customers
ALTER TABLE Customers
    ADD CONSTRAINT uq_customers_email UNIQUE (email),
    ADD CONSTRAINT chk_customers_email CHECK (email LIKE '%_@_%.__%'),
    ADD CONSTRAINT chk_customers_phone CHECK (phone IS NULL OR phone REGEXP '^[0-9]{10,15}$'),
    MODIFY registration_date DATE NOT NULL DEFAULT (CURRENT_DATE);
```

Pincodes

```
-- Pincodes
ALTER TABLE Pincodes
    ADD CONSTRAINT chk_pincode_format CHECK (pincode REGEXP '^[0-9]{6}$');
```

Addresses

```
-- Addresses
ALTER TABLE Addresses
    ADD CONSTRAINT uq_addresses_id_customer UNIQUE (address_id, customer_id),
    ADD CONSTRAINT chk_addresses_type CHECK (address_type IN ('Home','Work','Other')),
    MODIFY address_type VARCHAR(10) NOT NULL DEFAULT 'Home';
```

Categories

```
-- Categories
ALTER TABLE Categories
    ADD CONSTRAINT uq_categories_name UNIQUE (category_name);
```

Products

```
-- Products
ALTER TABLE Products
    ADD CONSTRAINT uq_products_name_brand UNIQUE (product_name, brand),
    ADD CONSTRAINT chk_products_price CHECK (price > 0),
    ADD CONSTRAINT chk_products_stock CHECK (stock_quantity >= 0),
    MODIFY stock_quantity INT NOT NULL DEFAULT 0;
```

Shopping_Cart

```
-- Shopping_Cart
ALTER TABLE Shopping_Cart
    ADD CONSTRAINT uq_cart_customer UNIQUE (customer_id),
    MODIFY created_date DATE NOT NULL DEFAULT (CURRENT_DATE);
```

Cart_Items

```
-- Cart_Items
ALTER TABLE Cart_Items
    ADD CONSTRAINT uq_cartitems_cart_product UNIQUE (cart_id, product_id),
    ADD CONSTRAINT chk_cartitems_qty CHECK (quantity > 0),
    MODIFY quantity INT NOT NULL DEFAULT 1;
```

Orders (composite FK: the delivery address must belong to the ordering customer)

```
-- Orders (composite FK: the delivery address must belong to the ordering customer)
ALTER TABLE Orders
    DROP FOREIGN KEY fk_orders_address,
    ADD CONSTRAINT fk_orders_address_customer FOREIGN KEY (address_id, customer_id)
        REFERENCES Addresses (address_id, customer_id),
    ADD CONSTRAINT chk_orders_status
        CHECK (order_status IN ('Placed','Shipped','Delivered','Cancelled')),
    ADD CONSTRAINT chk_orders_total CHECK (total_amount >= 0),
    MODIFY order_date   DATETIME    NOT NULL DEFAULT CURRENT_TIMESTAMP,
    MODIFY order_status VARCHAR(20) NOT NULL DEFAULT 'Placed';
```

Order_Items

```
-- Order_Items
ALTER TABLE Order_Items
    ADD CONSTRAINT uq_orderitems_order_product UNIQUE (order_id, product_id),
    ADD CONSTRAINT chk_orderitems_qty CHECK (quantity > 0),
    ADD CONSTRAINT chk_orderitems_price CHECK (unit_price >= 0);
```

Payments

```
-- Payments
ALTER TABLE Payments
    ADD CONSTRAINT uq_payments_order UNIQUE (order_id),
    ADD CONSTRAINT chk_payments_method
        CHECK (payment_method IN ('Card','UPI','Net Banking','Cash on Delivery')),
    ADD CONSTRAINT chk_payments_status
        CHECK (payment_status IN ('Pending','Successful','Failed')),
    ADD CONSTRAINT chk_payments_amount CHECK (amount_paid >= 0),
    MODIFY payment_status VARCHAR(20) NOT NULL DEFAULT 'Pending';
```

Shipments

```
-- Shipments
ALTER TABLE Shipments
    ADD CONSTRAINT uq_shipments_order UNIQUE (order_id),
    ADD CONSTRAINT uq_shipments_tracking UNIQUE (tracking_number),
    ADD CONSTRAINT chk_shipments_status
        CHECK (shipment_status IN ('In Transit','Delivered','Delayed')),
    ADD CONSTRAINT chk_shipments_dates CHECK (expected_delivery_date >= shipped_date),
    MODIFY shipment_status VARCHAR(20) NOT NULL DEFAULT 'In Transit';
```

Reviews

```
-- Reviews
ALTER TABLE Reviews
    ADD CONSTRAINT uq_reviews_customer_product UNIQUE (customer_id, product_id),
    ADD CONSTRAINT chk_reviews_rating CHECK (rating BETWEEN 1 AND 5),
    MODIFY review_date DATE NOT NULL DEFAULT (CURRENT_DATE);
```

3.1 Result of executing the script

SQL query

```
SELECT 'constraints applied' AS status;
```

Output

```
+----------------------+
```

```
| status              |
+---------------------+
| constraints applied |
+---------------------+
```

Explanation: The script ran without errors. The listing in Section 4 reads the constraints back from the data dictionary.

4. Constraint Inventory (from the data dictionary)

4.1 Number of constraints per table

SQL query

```
SELECT TABLE_NAME AS tbl,
  SUM(CONSTRAINT_TYPE='PRIMARY KEY') AS pk, SUM(CONSTRAINT_TYPE='FOREIGN KEY') AS fk,
  SUM(CONSTRAINT_TYPE='UNIQUE') AS uq, SUM(CONSTRAINT_TYPE='CHECK') AS chk
FROM information_schema.TABLE_CONSTRAINTS WHERE TABLE_SCHEMA='online_shopping'
GROUP BY TABLE_NAME ORDER BY TABLE_NAME;
```

Output

```
+---------------+------+------+------+------+
| tbl           | pk   | fk   | uq   | chk  |
+---------------+------+------+------+------+
| Addresses     |    1 |    2 |    1 |    1 |
| Cart_Items    |    1 |    2 |    1 |    1 |
| Categories    |    1 |    0 |    1 |    0 |
| Customers     |    1 |    0 |    1 |    2 |
| Orders        |    1 |    2 |    0 |    2 |
| Order_Items   |    1 |    2 |    1 |    2 |
| Payments      |    1 |    1 |    1 |    3 |
| Pincodes      |    1 |    0 |    0 |    1 |
| Products      |    1 |    1 |    1 |    2 |
| Reviews       |    1 |    2 |    1 |    1 |
| Shipments     |    1 |    1 |    2 |    2 |
| Shopping_Cart |    1 |    1 |    1 |    0 |
+---------------+------+------+------+------+
```

Explanation: Every table has a primary key; the counts show the foreign keys, unique keys and CHECK rules now active.

4.2 DEFAULT values defined

SQL query

```
SELECT TABLE_NAME AS tbl, COLUMN_NAME AS col, COLUMN_DEFAULT AS default_value
FROM information_schema.COLUMNS
WHERE TABLE_SCHEMA='online_shopping' AND COLUMN_DEFAULT IS NOT NULL
ORDER BY TABLE_NAME, ORDINAL_POSITION;
```

Output

```
+---------------+-------------------+---------------------+
| tbl           | col               | default_value       |
+---------------+-------------------+---------------------+
| Addresses     | address_type      | 'Home'              |
| Cart_Items    | quantity          | 1                   |
| Categories    | description       | NULL                |
| Customers     | phone             | NULL                |
| Customers     | registration_date | curdate()           |
| Orders        | order_date        | current_timestamp() |
| Orders        | order_status      | 'Placed'            |
| Payments      | payment_status    | 'Pending'           |
| Payments      | payment_date      | NULL                |
| Products      | description       | NULL                |
| Products      | stock_quantity    | 0                   |
| Products      | brand             | NULL                |
| Reviews       | comment           | NULL                |
| Reviews       | review_date       | curdate()           |
| Shipments     | shipment_status   | 'In Transit'        |
| Shopping_Cart | created_date      | curdate()           |
+---------------+-------------------+---------------------+
```

Explanation: These columns receive an automatic value when an INSERT omits them.

5. Verification of Integrity

Each test below inserts a small, valid **test fixture** inside a transaction, attempts the operation being tested, and finally issues **ROLLBACK**, so all tables are empty again afterwards. Error messages are the real messages returned by the DBMS.

Test fixture used by every test

```
START TRANSACTION;
INSERT INTO Pincodes VALUES ('560001','Bengaluru','Karnataka'),('400001','Mumbai','Maharashtra');
INSERT INTO Customers VALUES (1,'Test','User','test.user@example.com','9876543210','hash','2026-01-01'),
```

```
                          (2,'Other','Person','other@example.com',NULL,'hash','2026-01-02');
INSERT INTO Addresses VALUES (1,1,'1 Test Street','560001','Home'),(2,2,'2 Other Street','400001','Work');
INSERT INTO Categories VALUES (1,'Electronics','Test category');
INSERT INTO Products VALUES (1,1,'Test Phone','Test product',10000.00,5,'TestBrand');
INSERT INTO Shopping_Cart VALUES (1,1,'2026-01-03');
INSERT INTO Cart_Items VALUES (1,1,1,1);
INSERT INTO Orders VALUES (1,1,1,'2026-02-01 10:00:00','Placed',10000.00);
INSERT INTO Order_Items VALUES (1,1,1,1,10000.00);
INSERT INTO Payments VALUES (1,1,'UPI','Successful','2026-02-01 10:02:00',10000.00);
INSERT INTO Shipments VALUES (1,1,'Delhivery','DLV0000000001','2026-02-02','2026-02-05','In Transit');
INSERT INTO Reviews VALUES (1,1,1,5,'Great product','2026-02-10');
-- (each test ends with ROLLBACK)
```

5.1 PRIMARY KEY – entity integrity

Test 1 – duplicate primary key

Test SQL

```
INSERT INTO Categories VALUES (1, 'Books', 'Duplicate id');
```

Output

```
ERROR 1062 (23000): Duplicate entry '1' for key 'PRIMARY'
```

Explanation: Rejected (error 1062): category_id 1 already exists, so two rows can never share the same key.

5.2 FOREIGN KEY – referential integrity

Test 2 – child row with a non-existent parent

Test SQL

```
INSERT INTO Addresses VALUES (3, 99, '9 Ghost Lane', '560001', 'Home');
```

Output

```
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails (`online_shopping`.`Addresses`, CONSTRAINT
`fk_addresses_customer` FOREIGN KEY (`customer_id`) REFERENCES `Customers` (`customer_id`) ON DELETE CASCADE)
```

Explanation: Rejected (error 1452): there is no customer 99, so an address cannot be created for them.

Test 3 – deleting a parent that is still referenced (RESTRICT)

Test SQL

```
DELETE FROM Products WHERE product_id = 1;
SELECT COUNT(*) AS products_remaining FROM Products;
```

Output

```
+--------------------+
| products_remaining |
+--------------------+
|                  1 |
+--------------------+
ERROR 1451 (23000): Cannot delete or update a parent row: a foreign key constraint fails (`online_shopping`.`Order_Items`, CONSTRAINT
`fk_orderitems_product` FOREIGN KEY (`product_id`) REFERENCES `Products` (`product_id`))
```

Explanation: Rejected (error 1451): Order_Items still contains the product, so it cannot be deleted. Order history is protected.

Test 4 – deleting a customer who has orders

Test SQL

```
DELETE FROM Customers WHERE customer_id = 1;
SELECT COUNT(*) AS customers_remaining FROM Customers WHERE customer_id = 1;
```

Output

```
+---------------------+
| customers_remaining |
+---------------------+
|                   1 |
+---------------------+
ERROR 1451 (23000): Cannot delete or update a parent row: a foreign key constraint fails (`online_shopping`.`Orders`, CONSTRAINT
`fk_orders_address_customer` FOREIGN KEY (`address_id`, `customer_id`) REFERENCES `Addresses` (`address_id`, `customer_id`))
```

Explanation: Rejected (error 1451): the customer's address is used by an order, so the CASCADE to Addresses is blocked and the customer remains.

Test 5 – ON DELETE CASCADE removes dependent rows

Test SQL

```
SELECT COUNT(*) AS items_before FROM Cart_Items WHERE cart_id = 1;
DELETE FROM Shopping_Cart WHERE cart_id = 1;
SELECT COUNT(*) AS items_after FROM Cart_Items WHERE cart_id = 1;
```

Output

```
+--------------+
| items_before |
+--------------+
|            1 |
+--------------+
+-------------+
| items_after |
+-------------+
|           0 |
+-------------+
```

Explanation: Deleting a cart automatically deletes its items (1 → 0), leaving no orphan rows.

Test 6 – composite FK: an order must use the customer's own address

Test SQL

```
INSERT INTO Orders VALUES (2, 2, 1, '2026-03-01 09:00:00', 'Placed', 500.00);
```

Output

```
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails (`online_shopping`.`Orders`, CONSTRAINT
`fk_orders_address_customer` FOREIGN KEY (`address_id`, `customer_id`) REFERENCES `Addresses` (`address_id`, `customer_id`))
```

Explanation: Rejected (error 1452): address 1 belongs to customer 1, not customer 2. Without the composite foreign key this inconsistent row would have been accepted.

5.3 UNIQUE

Test 7 – duplicate e-mail address

Test SQL

```
INSERT INTO Customers VALUES (3, 'Dup', 'Mail', 'test.user@example.com', NULL, 'hash', '2026-01-05');
```

Output

```
ERROR 1062 (23000): Duplicate entry 'test.user@example.com' for key 'uq_customers_email'
```

Explanation: Rejected (error 1062, uq_customers_email): every customer must have a distinct login e-mail.

Test 8 – 1:1 relationships (second cart, second payment, second shipment for the same owner)

Test SQL

```
INSERT INTO Shopping_Cart VALUES (2, 1, '2026-01-04');
INSERT INTO Payments VALUES (2, 1, 'Card', 'Pending', NULL, 0);
INSERT INTO Shipments VALUES (2, 1, 'Blue Dart', 'BD0000000002', '2026-02-02', '2026-02-06', 'In Transit');
```

Output

```
ERROR 1062 (23000): Duplicate entry '1' for key 'uq_cart_customer'
ERROR 1062 (23000): Duplicate entry '1' for key 'uq_payments_order'
ERROR 1062 (23000): Duplicate entry '1' for key 'uq_shipments_order'
```

Explanation: All three are rejected: UNIQUE keys on customer_id and order_id enforce “one active cart per customer” and “one payment / one shipment per order”.

Test 9 – duplicate line items and duplicate reviews

Test SQL

```
INSERT INTO Cart_Items VALUES (2, 1, 1, 2);
INSERT INTO Order_Items VALUES (2, 1, 1, 1, 10000.00);
INSERT INTO Reviews VALUES (2, 1, 1, 4, 'Second review', '2026-02-12');
```

Output

```
ERROR 1062 (23000): Duplicate entry '1-1' for key 'uq_cartitems_cart_product'
ERROR 1062 (23000): Duplicate entry '1-1' for key 'uq_orderitems_order_product'
ERROR 1062 (23000): Duplicate entry '1-1' for key 'uq_reviews_customer_product'
```

Explanation: Rejected: the same product cannot appear twice in one cart or order (increase the quantity instead), and a customer can review a product only once.

5.4 CHECK

Test 10 – invalid prices, stock and quantities

Test SQL

```
INSERT INTO Products VALUES (2, 1, 'Bad Price', NULL, -50.00, 5, 'X');
INSERT INTO Products VALUES (3, 1, 'Bad Stock', NULL, 99.00, -1, 'X');
INSERT INTO Cart_Items VALUES (3, 1, 1, 0);
```

Output

```
ERROR 4025 (23000): CONSTRAINT `chk_products_price` failed for `online_shopping`.`Products`
ERROR 4025 (23000): CONSTRAINT `chk_products_stock` failed for `online_shopping`.`Products`
ERROR 4025 (23000): CONSTRAINT `chk_cartitems_qty` failed for `online_shopping`.`Cart_Items`
```

Explanation: All rejected: price must be positive, stock cannot be negative, and quantity must be at least 1.

Test 11 – invalid rating, status values and address type

Test SQL

```
INSERT INTO Reviews VALUES (3, 1, 2, 6, 'Rating out of range', '2026-02-12');
UPDATE Orders SET order_status = 'Teleported' WHERE order_id = 1;
INSERT INTO Addresses VALUES (3, 1, '3 Test Street', '560001', 'Castle');
```

Output

```
ERROR 4025 (23000): CONSTRAINT `chk_reviews_rating` failed for `online_shopping`.`Reviews`
ERROR 4025 (23000): CONSTRAINT `chk_orders_status` failed for `online_shopping`.`Orders`
ERROR 4025 (23000): CONSTRAINT `chk_addresses_type` failed for `online_shopping`.`Addresses`
```

Explanation: All rejected: rating must be 1–5, order_status must be Placed/Shipped/Delivered/Cancelled, and address_type must be Home/Work/Other.

Test 12 – invalid formats and dates

Test SQL

```
INSERT INTO Customers VALUES (4, 'Bad', 'Phone', 'bad.phone@example.com', 'ABC123', 'hash', '2026-01-05');
INSERT INTO Pincodes VALUES ('12AB', 'Nowhere', 'None');
UPDATE Shipments SET expected_delivery_date = '2026-01-01' WHERE shipment_id = 1;
```

Output

```
ERROR 4025 (23000): CONSTRAINT `chk_customers_phone` failed for `online_shopping`.`Customers`
ERROR 4025 (23000): CONSTRAINT `chk_pincode_format` failed for `online_shopping`.`Pincodes`
ERROR 4025 (23000): CONSTRAINT `chk_shipments_dates` failed for `online_shopping`.`Shipments`
```

Explanation: All rejected: phone must be 10–15 digits, a PIN code must be 6 digits, and the expected delivery date cannot be earlier than the shipped date.

5.5 DEFAULT and NOT NULL

Test 13 – default values are filled in automatically

Test SQL

```
INSERT INTO Customers (customer_id, first_name, last_name, email, password_hash)
       VALUES (5, 'Default', 'Demo', 'default.demo@example.com', 'hash');
INSERT INTO Orders (order_id, customer_id, address_id, total_amount) VALUES (3, 1, 1, 250.00);
INSERT INTO Payments (payment_id, order_id, payment_method, amount_paid) VALUES (3, 3, 'UPI', 0);
SELECT registration_date = CURDATE() AS reg_is_today FROM Customers WHERE customer_id = 5;
SELECT order_status, DATE(order_date) = CURDATE() AS ordered_today FROM Orders WHERE order_id = 3;
SELECT payment_status FROM Payments WHERE payment_id = 3;
```

Output

```
+--------------+
| reg_is_today |
+--------------+
|            1 |
+--------------+
+--------------+---------------+
| order_status | ordered_today |
+--------------+---------------+
| Placed       |             1 |
+--------------+---------------+
+----------------+
| payment_status |
+----------------+
```

```
| Pending         |
+-----------------+
```

Explanation: None of the defaulted columns were supplied, yet registration_date is today, order_status is 'Placed', order_date is now and payment_status is 'Pending'.

Test 14 – NOT NULL

Test SQL

```
INSERT INTO Customers VALUES (6, 'No', 'Email', NULL, NULL, 'hash', '2026-01-05');
```

Output

```
ERROR 1048 (23000): Column 'email' cannot be null
```

Explanation: Rejected (error 1048): e-mail is mandatory.

5.6 Data is clean after testing

All tables are empty after the rollbacks, ready for Assignment 7

SQL query

```
SELECT (SELECT COUNT(*) FROM Customers) AS customers, (SELECT COUNT(*) FROM Addresses) AS addresses,
       (SELECT COUNT(*) FROM Products) AS products, (SELECT COUNT(*) FROM Orders) AS orders,
       (SELECT COUNT(*) FROM Payments) AS payments, (SELECT COUNT(*) FROM Reviews) AS reviews;
```

Output

```
+-----------+-----------+----------+--------+----------+---------+
| customers | addresses | products | orders | payments | reviews |
+-----------+-----------+----------+--------+----------+---------+
|         0 |         0 |        0 |      0 |        0 |       0 |
+-----------+-----------+----------+--------+----------+---------+
```

Explanation: Every test row was rolled back, so no test data leaks into the real data set.

6. Conclusion

All constraint types required by the assignment are implemented and verified: primary keys, single and composite foreign keys with CASCADE/RESTRICT rules, UNIQUE keys (including those enforcing one-to-one relationships), CHECK rules for value domains and formats, and DEFAULT values. The tests show that duplicate keys, orphan rows, invalid values and cross-customer address errors are rejected by the DBMS itself, so the integrity of the Online Shopping data does not depend on application code alone.